

Python

高效复习手册

专为有 C 语言基础、Python 生疏的学习者设计

精简 · 重点突出 · 即查即用

1. Python vs C -- 10分钟思维切换

你把下面这张表记住，Python 就算[捡回来]一半了。学 C 时你习惯管理内存、声明类型、用指针；Python 把这些都藏起来了 -- 你只管写逻辑。

C vs Python 速查表

项目	C	Python
类型声明	<code>int x = 10;</code>	<code>x = 10</code> # 无需声明
语句结束	需要分号 ;	不需要分号
代码块	花括号 { }	缩进(4空格)
逻辑运算	<code>&& !</code>	<code>and or not</code>
相等判断	<code>==</code>	<code>==</code>
不等判断	<code>!=</code>	<code>!=</code>
布尔值	0/非0 (无true/false)	True/False (首字母大写)
空值	NULL	None
常量	<code>#define / const</code>	惯例大写 = 约定俗成
注释	<code>//单行 /*多行*/</code>	<code>#单行 '''多行'''</code>
整数除法	<code>5/2 = 2</code> (整除)	<code>5/2 = 2.5</code> <code>5//2 = 2</code>
字符串	<code>char s[] = "...";</code>	<code>s = '...'</code> 或 <code>"..."</code> 无区别
数组/列表	<code>int arr[10];</code> (定长)	<code>lst = [1, 2, 3]</code> (动态)
循环	<code>for (i=0; i<n; i++)</code>	<code>for i in range(n):</code>

核心心态：

Python

是[鸭子类型]

--

如果你走起来像鸭子、叫起来像鸭子，那你就是鸭子。变量不需要声明类型，运行时才确定。这跟

C

的静态类型完全不同，写 Python 时先忘掉 `int *p` 这种东西。

2. 基础数据类型 & 运算

2.1 数字 (int / float / complex)

```
a = 10          # int    任意精度, 不存在 overflow
b = 3.14        # float  双精度 (64-bit)
c = 1 + 2j      # complex 工程计算用

# 常用运算
5 / 2           # = 2.5    (真除法, C 程序员注意!)
5 // 2          # = 2      (地板除 = C 的整数除法)
5 % 2           # = 1      (取模)
5 ** 3          # = 125    (幂运算, 比 pow() 直观)
round(3.1415, 2) # = 3.14  (四舍五入)

# 类型转换
int("123")      # 123
float("3.14")   # 3.14
str(100)        # "100"
```

Python 的 int 不会溢出 -- 它会自动扩展精度。这是和 C 最大的不同。

2.2 字符串 (str)

```
s = 'hello'     # 单引号/双引号无区别, 选一种保持统一即可
s = "it's ok"   # 内含单引号时用双引号包裹

# 核心操作 (这是你用得最多的)
len(s)          # 5        -- 长度
s[0]            # 'h'      -- 索引 (从0开始)
s[-1]           # 'o'      -- 倒数第一个
s[0:3]          # 'hel'    -- 切片 [start:end) 左闭右开
s[::2]          # 'hlo'    -- 步长为2
"hello " + "world" # 拼接
f"1+1={1+1}"    # f-string -- 格式化首选! (Python 3.6+)
"{ } + { }".format(1, 2) # format -- 旧式但常见
", ".join(["a", "b", "c"]) # "a,b,c"
"a,b,c".split(", ")      # ['a', 'b', 'c']
" x ".strip()            # "x"          -- 去两端空白
"hello".upper()          # "HELLO"
"hello".replace("l", "L") # "heLLo"
"12,45,78".find("45")    # 3            -- 子串位置, 找不到返回 -1
```

重点: f-string 是写 Python 最常用的格式化方式。f"名字: {name}, 年龄: {age}" -- 变量直接嵌入, 不记占位符。

2.3 布尔 (bool)

```
flag = True    # 首字母大写! 不是 true 也不是 TRUE
flag = False
```

```
# 逻辑运算用英文单词 (不用 && || !)
```

```
a and b        # C: a && b
```

```
a or b         # C: a || b
```

```
not a          # C: !a
```

```
# 以下都是 False (其他全是 True)
```

```
bool(0)        # False
```

```
bool(0.0)      # False
```

```
bool("")       # False  -- 空字符串
```

```
bool([])       # False  -- 空列表
```

```
bool(None)     # False"
```

3. 核心容器类型（重点中的重点）

Python 有4大容器，其中最常用的是 List 和 Dict。如果你只记住这两个，也足够覆盖80%的日常编程。这些容器都是[引用语义] -- 和 C 的数组不同，它们存储的是对象的引用。

3.1 列表 List -- 可变、有序、可重复

```
# 创建
lst = [1, 2, 3, 4]
lst = list(range(10))      # [0, 1, 2, ..., 9]
lst = [0] * 5               # [0, 0, 0, 0, 0] （注意：对象会被共享!）

# 增删改查 -- 最常用操作
lst.append(5)               # [1, 2, 3, 4, 5]      尾部添加
lst.insert(1, 99)           # [1, 99, 2, 3, 4, 5]   指定位置插入
lst.pop()                   # 弹出最后一个，返回5
lst.pop(1)                  # 弹出索引1的元素
lst.remove(3)                # 删除第一个值为3的元素
lst[0] = 100                # 修改
lst.index(4)                # 查找4的索引
lst.count(2)                # 统计2出现次数
lst.sort()                  # 原地排序
lst.sort(reverse=True)      # 降序
sorted(lst)                 # 返回新列表(不改变原列表)

# 遍历
for item in lst:             # 逐个取元素
    print(item)
for i, item in enumerate(lst): # 同时要索引+值 -- enumerate!
    print(i, item)
# 列表推导式（见第9节）是操作列表的利器"
```

3.2 元组 Tuple -- 不可变、有序

```
t = (1, 2, 3)               # 创建后不能修改（只读列表）
t = 1, 2, 3                 # 括号可以省略
single = (1,)               # 单元素元组：逗号不能省！不是(1)

# 常用于：函数返回多个值
def get_pos():
    return 100, 200          # 实际返回 tuple: (100, 200)
x, y = get_pos()             # 自动解包"
```

3.3 字典 Dict -- 键值对（哈希表）

```

d = {"name": "Tom", "age": 20}
d = dict(name="Tom", age=20)    # 另一种创建方式

# 核心操作
d["name"]          # "Tom"          -- 取值 (KeyError if not exist)
d.get("name")      # "Tom"          -- 取值 (不存在返回 None, 更安全)
d.get("x", 0)       # 0              -- 不存在时返回默认值
d["score"] = 95     # 添加/修改
d.pop("age")        # 删除并返回
d.keys()            # dict_keys(['name', 'score'])
d.values()          # dict_values(['Tom', 95])
d.items()           # dict_items([('name', 'Tom'), ('score', 95)])

# 遍历 -- 最常用范式
for k, v in d.items():
    print(f"{k} -> {v}")

# 检查键是否存在
if "name" in d:     # 用 in, 不是 has_key()
    ...

```

3.4 集合 Set -- 无序、不重复

```

s = {1, 2, 3}
s.add(4)          # 添加
s.remove(2)       # 删除
s1 & s2            # 交集
s1 | s2           # 并集
s1 - s2           # 差集

# 去重利器
lst = [1, 2, 2, 3, 3, 3]
unique = list(set(lst)) # [1, 2, 3]

```

4. 控制流

4.1 条件分支

```
# 缩进是语法的一部分！不是可选的！
if x > 0:
    print("正数")
elif x == 0:      # 注意：是 elif 不是 else if
    print("零")
else:
    print("负数")

# 三元表达式 (C: x > 0 ? "正" : "负")
result = "正" if x > 0 else "负"
```

4.2 循环

```
# ---- for: 遍历可迭代对象（最常用） ----
for i in range(5):      # i = 0, 1, 2, 3, 4    (左闭右开)
    print(i)
for i in range(2, 6):   # i = 2, 3, 4, 5
    print(i)
for i in range(0, 10, 2): # i = 0, 2, 4, 6, 8    (步长)
    print(i)

# 遍历列表
for item in lst:        # 直接取元素，不需要下标
    print(item)

# 同时要索引和值
for i, val in enumerate(lst):
    print(i, val)

# ---- while: 条件循环 ----
while x > 0:
    x -= 1

# ---- break / continue (和C一样) ----
for i in range(10):
    if i == 5:
        break      # 退出整个循环
    if i % 2 == 0:
        continue   # 跳过本次，进入下一次"
```

`range(n)` 是左闭右开 `[0, n)`，和切片一样。`for` 循环不需要索引变量 -- 直接遍历可迭代对象。C 程序员容易写的 `for i in range(len(lst)): print(lst[i])` 是反模式！应该写 `for item in lst: print(item)`。

5. 函数

5.1 def -- 定义函数

```
def add(a, b):
    """返回两数之和""" # docstring (文档字符串)
    return a + b

# 多返回值 (实际是返回 tuple)
def get_min_max(lst):
    return min(lst), max(lst) # 返回元组
a, b = get_min_max([1, 2, 3, 4]) # 解包接收"
```

5.2 参数类型 (必掌握)

```
# 1. 位置参数 (必需参数)
def f(a, b):
    return a + b

# 2. 默认参数 (有默认值的放后面!)
def f(a, b=10):
    return a + b

# 陷阱: 默认值只计算一次! 不要用可变对象作默认值
# 错误做法: def f(lst=[]): <- [] 会被所有调用共享
# 正确做法: def f(lst=None): if lst is None: lst = []

# 3. 关键字参数 (调用时指定形参名)
def f(a, b, c):
    ...

f(a=1, b=2, c=3) # 顺序可打乱
f(1, c=3, b=2) # 混合使用

# 4. 可变参数 *args (打包为 tuple)
def f(*args):
    for a in args:
        print(a)
f(1, 2, 3) # args = (1, 2, 3)

# 5. 关键字可变参数 **kwargs (打包为 dict)
def f(**kwargs):
    for k, v in kwargs.items():
        print(k, v)
f(name="Tom", age=20) # kwargs = {'name': 'Tom', 'age': 20}

# 6. 参数顺序规则 (定义时)
# def f(pos, default=1, *args, keyword_only, **kwargs):
#     位置    默认    可变    仅关键字    关键字可变"
```

5.3 lambda -- 匿名函数 (单表达式)


```
# 语法: lambda 参数: 返回值表达式
add = lambda a, b: a + b
square = lambda x: x ** 2
# 常用于 sorted/map/filter 的 key 参数
lst = [("Tom", 20), ("Jerry", 18)]
lst.sort(key=lambda x: x[1]) # 按年龄排序"
```

5.4 装饰器 (decorator) -- 了解

```
# 本质: 把函数作为参数, 返回一个增强后的新函数
def timer(func):
    import time
    def wrapper(*args, **kwargs):
        t0 = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} cost: {time.time()-t0:.4f}s")
        return result
    return wrapper

@timer # 等价于: my_func = timer(my_func)
def my_func():
    ..."
```

6. 类与面向对象（实用最小集）

```
class Dog:
    species = "Canis"          # 类属性（所有实例共享）

    def __init__(self, name):   # 构造方法（不是 new，是 __init__）
        self.name = name       # 实例属性（每个实例独有）

    def bark(self):             # 实例方法（第一个参数必须是 self）
        print(f"{self.name}: Woof!")

# 使用
d = Dog("Buddy")               # 创建实例（没有 new 关键字）
d.bark()                       # "Buddy: Woof!"
Dog.species                    # "Canis"
```

6.1 继承

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        pass

class Cat(Animal):             # 继承
    def __init__(self, name, color):
        super().__init__(name) # 调用父类 __init__
        self.color = color
    def speak(self):
        return f"Meow, I'm {self.name}"
```

6.2 魔法方法（双下划线方法）

```
# 你不需要全部记住，这几个最常见：
__init__(self)      # 构造函数
__str__(self)       # str(obj) 时调用，面向用户
__repr__(self)      # repr(obj) 时调用，面向开发者/调试
__len__(self)       # len(obj) 时调用
__eq__(self, other) # obj1 == obj2 时调用
__getitem__(self, i) # obj[i] 时调用（让你的对象支持索引）
__enter__/__exit__  # with 语句（上下文管理器）"
```

和 C/C++ 最大的不同：Python 没有真正的 `private`。约定俗成的[私有]是名字前加一个下划线 `_secret`，加两个下划线 `__name` 会触发名称修饰(name mangling)，但也不是真正的私有。靠自觉，不靠编译器。

7. 文件操作

```
# 推荐方式 -- with 自动关闭文件 (不用自己 f.close())
with open("data.txt", "r", encoding="utf-8") as f:
    content = f.read()          # 读全部
    # 或
    for line in f:              # 逐行读 (大文件推荐)
        print(line.strip())

with open("out.txt", "w", encoding="utf-8") as f:
    f.write("Hello")           # 覆盖写入

with open("out.txt", "a", encoding="utf-8") as f:
    f.write("追加内容")        # 追加写入

# 模式速查
# "r"   -- 只读 (文件必须存在)
# "w"   -- 只写 (清空已有内容, 不存在则新建)
# "a"   -- 追加 (在末尾添加, 不存在则新建)
# "rb"  -- 二进制读 (图片/音频等)
# "wb"  -- 二进制写"
```

8. 异常处理

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"不能除以零: {e}")
except (ValueError, TypeError):    # 捕获多种
    print("值错误或类型错误")
except Exception as e:            # 捕获所有 (兜底)
    print(f"未知错误: {e}")
else:                              # try成功时执行 (可选)
    print("一切正常")
finally:                           # 无论是否异常都执行 (常用做清理)
    f.close()

# 抛异常
raise ValueError("这是自定义错误信息")

# 自定义异常
class MyError(Exception):
    pass
raise MyError("出事了")"
```

9. 推导式 (Comprehension) -- Python 的 [一句话魔法]

这是 Python 最优雅的特性之一。一行代码替代 for 循环，可读性强且效率高。

9.1 列表推导式 (最常用)

```
# 基本: [表达式 for 变量 in 可迭代对象]
squares = [x**2 for x in range(10)]      # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# 带过滤: [表达式 for 变量 in 可迭代对象 if 条件]
evens = [x for x in range(20) if x % 2 == 0] # 偶数

# 带变换:
words = ["hello", "world", "python"]
upper_words = [w.upper() for w in words]    # 全部大写
lengths = [len(w) for w in words]          # [5, 5, 6]

# if-else 在三元表达式中 (位置不同!)
vals = [x if x > 0 else 0 for x in [-2, 3, -1, 5]]
# vals = [0, 3, 0, 5]"
```

9.2 字典推导式

```
# {键表达式: 值表达式 for ... in ...}
squares_dict = {x: x**2 for x in range(5)}
# {0:0, 1:1, 2:4, 3:9, 4:16}

# 翻转键值
d = {"a": 1, "b": 2, "c": 3}
reversed_d = {v: k for k, v in d.items()}
# {1:'a', 2:'b', 3:'c'}"
```

9.3 集合推导式

```
# {表达式 for ... in ...}
unique_lengths = {len(w) for w in ["a", "ab", "abc", "a"]}
# {1, 2, 3}"
```

10. 常用内置函数速查

```
# ---- 转换 ----
int("42")      -> 42          str(42)      -> "42"
float("3.14")   -> 3.14       bool([])    -> False
list("abc")     -> ['a', 'b', 'c'] tuple([1, 2, 3]) -> (1, 2, 3)
bin(10)         -> '0b1010'   hex(255)    -> '0xff'
ord('A')        -> 65         chr(65)     -> 'A'

# ---- 数学 ----
abs(-5)         -> 5          pow(2, 3)   -> 8
round(3.141, 2) -> 3.14       max(1, 5, 3) -> 5
min(1, 5, 3)    -> 1          sum([1, 2, 3]) -> 6
divmod(10, 3)   -> (3, 1)     # 商+余

# ---- 序列操作 ----
len([1, 2, 3])  -> 3          range(5)    -> 0..4
sorted([3, 1, 2]) -> [1, 2, 3] reversed([1, 2, 3]) -> 迭代器
zip([1, 2], ['a', 'b']) -> [(1, 'a'), (2, 'b')]
enumerate(['a', 'b', 'c']) -> (0, 'a'), (1, 'b'), (2, 'c')

# ---- 过滤/映射 ----
filter(lambda x: x>0, [-1, 2, -3, 4]) -> [2, 4]
map(lambda x: x**2, [1, 2, 3]) -> [1, 4, 9]

# ---- 判断 ----
isinstance(42, int) -> True    type(42)      -> <class 'int'>
hasattr(obj, 'name') -> True/False callable(func) -> True/False
any([0, 0, 1])      -> True    all([1, 2, 0]) -> False

# ---- 实用 ----
print(*[1, 2, 3])    # 1 2 3 (解包打印)
input("提示:")        # 读取用户输入 (返回 str)
open("f.txt")         # 打开文件"
```

11. 实用代码片段（记住这几个就够了）

读取文件所有行 -> 处理 -> 写回

```
with open("in.txt", "r") as f:
    lines = [line.strip() for line in f if line.strip()]
processed = [line.upper() for line in lines]
with open("out.txt", "w") as f:
    f.write("\n".join(processed))"
```

统计列表元素频率

```
from collections import Counter
freq = Counter(['a', 'b', 'a', 'c', 'a', 'b'])
print(freq)           # Counter({'a':3, 'b':2, 'c':1})
print(freq.most_common(2)) # [('a', 3), ('b', 2)]"
```

字典按值排序

```
d = {"a": 3, "b": 1, "c": 2}
sorted_d = dict(sorted(d.items(), key=lambda x: x[1]))
# {'b':1, 'c':2, 'a':3}"
```

遍历目录下所有文件

```
import os
for root, dirs, files in os.walk("./my_folder"):
    for f in files:
        print(os.path.join(root, f))"
```

JSON 读写

```
import json
# 写
with open("data.json", "w") as f:
    json.dump({"name": "Tom", "age": 20}, f, indent=2, ensure_ascii=False)
# 读
with open("data.json", "r") as f:
    data = json.load(f)
print(data["name"])    # "Tom"
```

一行代码交换变量 / 同时赋值

```
a, b = b, a           # 交换
x = y = z = 0         # 链式赋值"
```

切片操作图解

```
lst = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
lst[2:5]    -> [2, 3, 4]        # 索引2到5(不含5)
lst[:3]     -> [0, 1, 2]        # 开头到索引3(不含)
lst[7:]     -> [7, 8, 9]        # 索引7到结尾
lst[-3:]    -> [7, 8, 9]        # 倒数3个
lst[::-1]   -> [9, 8, ..., 0]    # 反转
lst[::2]    -> [0, 2, 4, 6, 8]  # 每隔一个"
```

复习建议

1. 每天花15分钟过一遍第1节(C vs Python速查表)，一周就能形成条件反射。
2. 第3节(List/Dict)和第9节(推导式)是写Python最常用的部分，优先掌握。
3. 不要死记语法 -- 拿个小项目(比如这个温控系统)用Python重写一遍，比看十遍都管用。
4. 写代码时把这个PDF放旁边当字典用，而不是当教材读。

-- 祝顺利 :)